# Project Documentation

## Project Title

Spend Wise- A Simple Backend Learning Project using Node.js,Express,MongoDB and JWT

## Introduction

- **Project Title:** Spend Wise
- **Team ID:** SWTID-2026-9455
- **Team Leader**: Aafrin J (j.aafrin06@gmail.com)

- **Team Members**:
  1. Aishwarya S (aishwarya7487@gmail.com)
  2. Rakkammal P (rakkammal2007@gmail.com)
  3. RosanFathima M (roshanfathima2006@gmail.com)
  4. Akila S (akilaas2006@gmail.com)

## Project Overview

Spend Wise is a backend application designed to help beginner developers learn how to build secure REST APIs using Node.js, Express, MongoDB, and JWT authentication. The project focuses on implementing user authentication and basic CRUD operations in a clean and simple way so students can understand how backend systems work in real-world scenarios.

**Key Objectives:**

- Provide secure user authentication using JWT
- Enable basic CRUD operations on user data (transactions/records)
- Demonstrate MongoDB integration using Mongoose
- Show how protected routes work using middleware
- Maintain a simple and beginner-friendly backend architecture

**Target Users:**

- Beginner backend developers
- Students learning Node.js and Express
- Freshers preparing for technical interviews
- Anyone who wants a simple project to understand JWT and MongoDB
- Learners who want hands-on practice with REST APIs

# Project Instruction

SpendWise is a backend-only project built using Node.js and Express.

**Quick Start:**

npm init

npm install

npm run dev

**Steps to Create the Project:**

- Choose or create a directory for the backend project.
- Open terminal and navigate to that directory.
- Initialize Node.js project using `npm init`.
- Install required packages: express, mongoose, jsonwebtoken, bcrypt, cors, dotenv.
- Create folders: models, routes, controllers, middleware, config.
- Start the server using `npm run dev`.
- Test APIs using Postman or Thunder Client at `http://localhost:5000`.

# Pre-requisites:

### Software Requirements

| Software | Purpose |
|---|---|
| Node.js | JavaScript runtime for backend |
| MongoDB | NoSQL database for data storage |
| npm | Package manager |
| Git | Version control system |
| Code Editor | VS Code or similar |

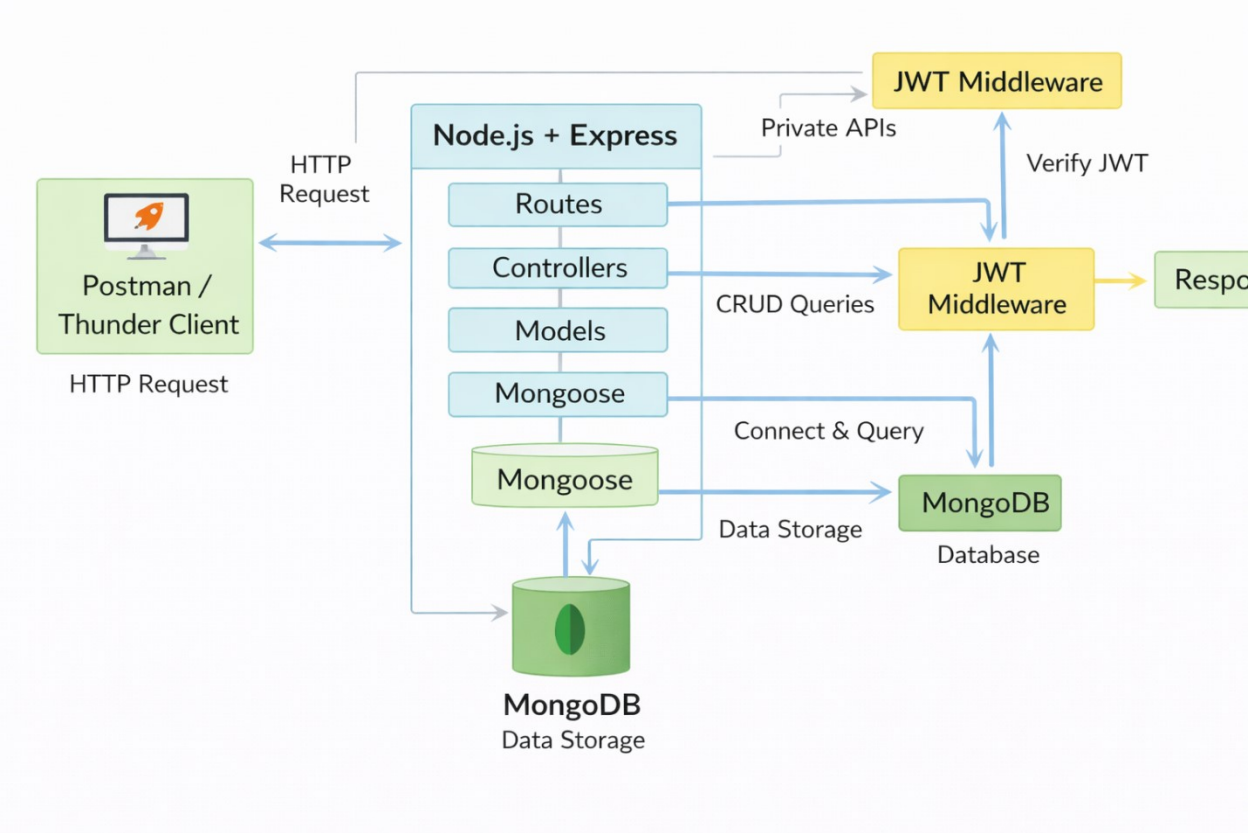
### Knowledge Requirements

- JavaScript ES6+
- Node.js and Express.js
- MongoDB and Mongoose
- JWT authentication
- Basic REST API concepts
- Git basics

# Thumbnail for Project

- A simple backend project thumbnail representing:
  **Node.js + Express + MongoDB + JWT + CRUD APIs**

# Project Architecture Image




# Technical Architecture

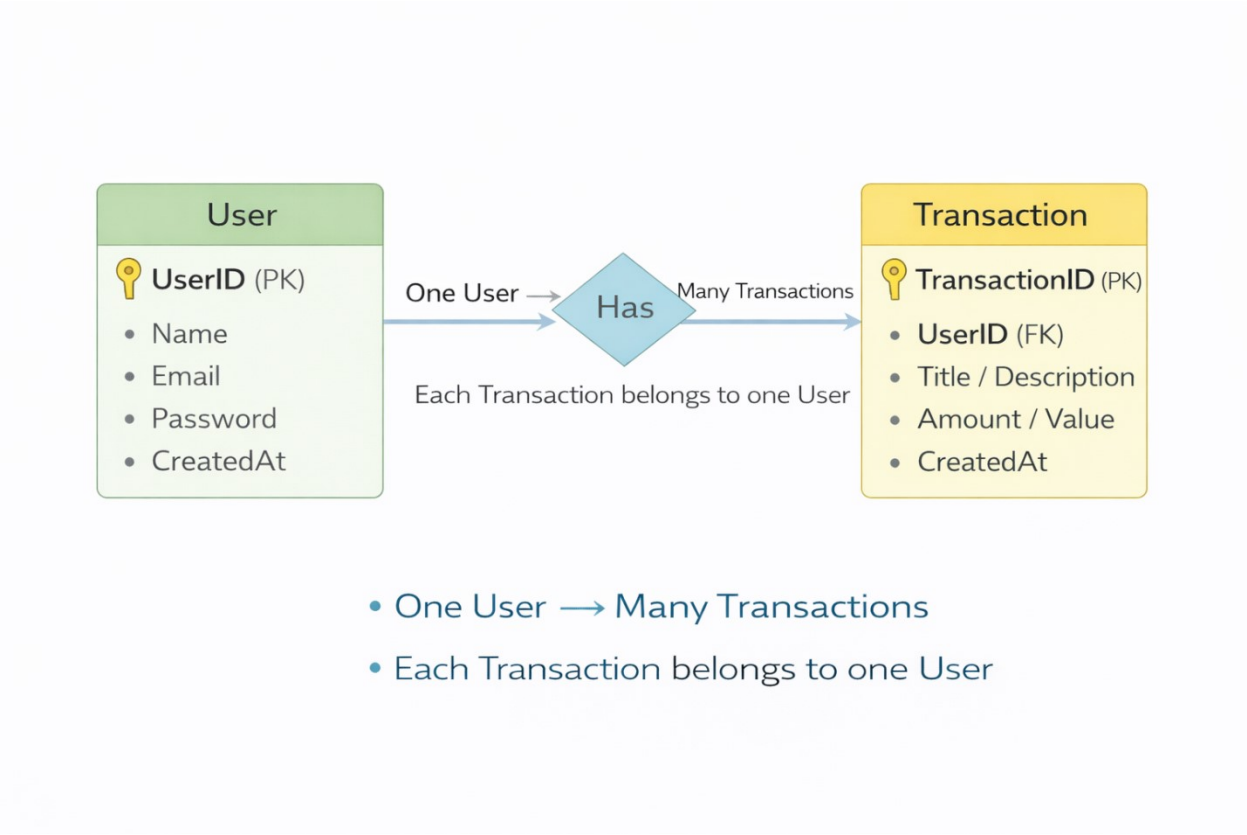
### Backend Architecture (Node.js + Express)

- **Controllers:** Handle business logic for authentication and CRUD
- **Models:** Mongoose schemas for User and Transaction
- **Routes:** API endpoints for auth and CRUD operations
- **Middleware:** JWT verification and request protection
- **Configuration:** Database connection and environment variables

### Database Architecture (MongoDB)

- Users Collection:
  `_id, name, email, password (hashed), createdAt`
- Transactions Collection:
  `_id, userId (FK), title/description, amount/value, createdAt`
- Relationship:
  Many Transactions belong to One User

## ER Diagram




## Features

| Feature | Description |
|---|---|
| User Authentication | JWT-based registration and login |
| CRUD Operations | Create, Read, Update, Delete transactions |
| Protected Routes | Secure APIs using JWT middleware |

| | |
|---|---|
| MongoDB Integration | Store and fetch data using Mongoose |

## Roles and Responsibilities

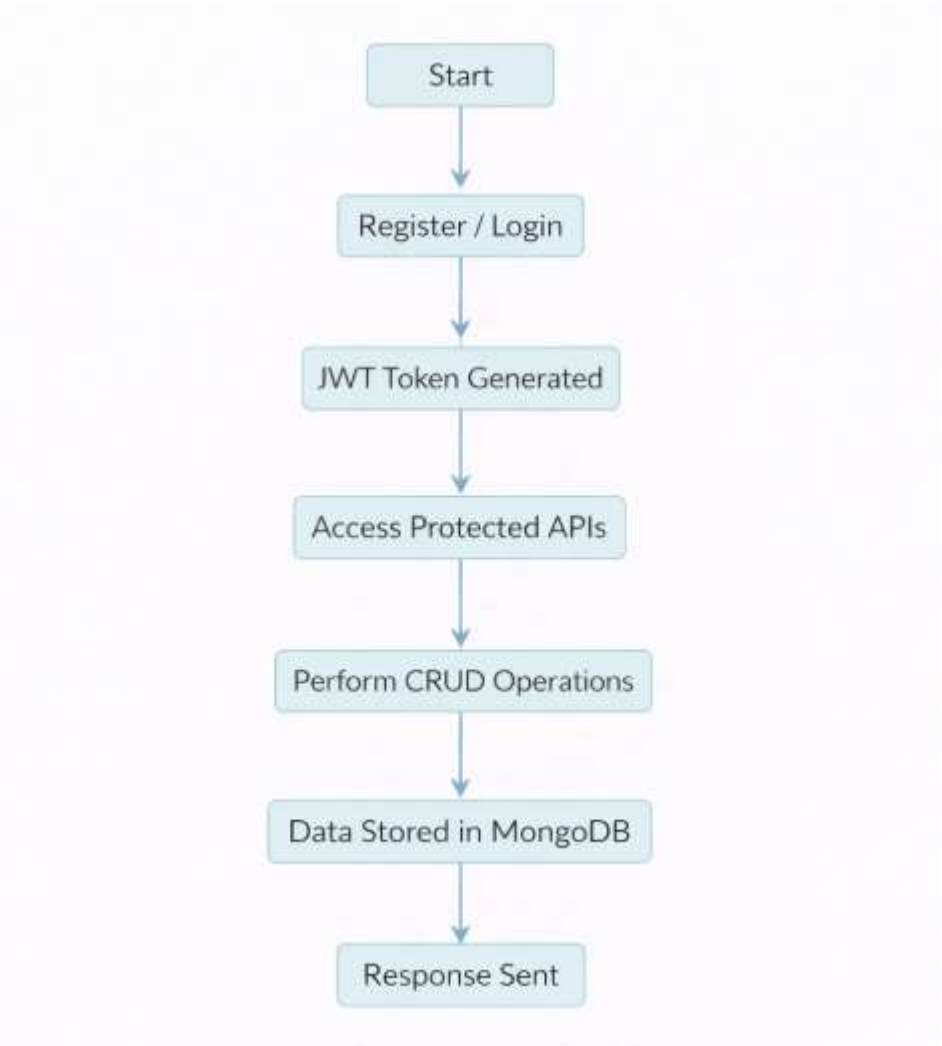
Student / Developer:

- Write backend code
- Create APIs
- Test APIs using Postman/Thunder Client
- Push code to GitHub

System (Backend):

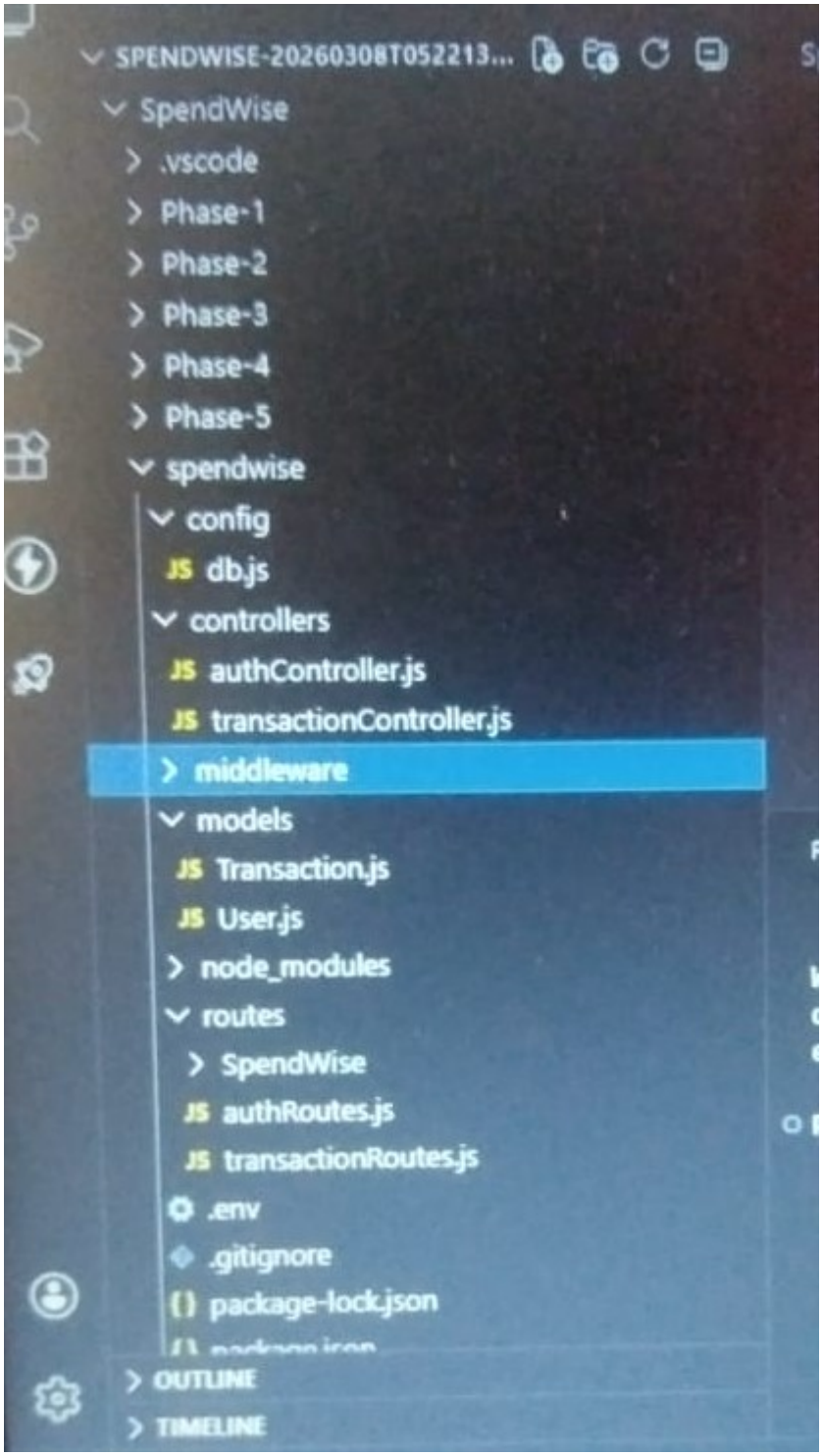
- Authenticate users using JWT
- Handle CRUD operations
- Store and fetch data from MongoDB
- Protect private routes

## Users Flow:


Start
Register / Login
JWT Token Generated
Access Protected APIs
Perform CRUD Operations
Data Stored in MongoDB
Response Sent

# PROJECT SETUP AND CONFIGURATION

## Folder Structure



## Project Setup

1. Install required tools:

- Node.js
- MongoDB

2. Create project folders:
   - server folders (models, routes, controllers, middleware, config)

3. Install Backend Packages:
   - Express
   - Mongoose
   - Cors
   - Jsonwebtoken
   - Bcrypt
   - Dotenv

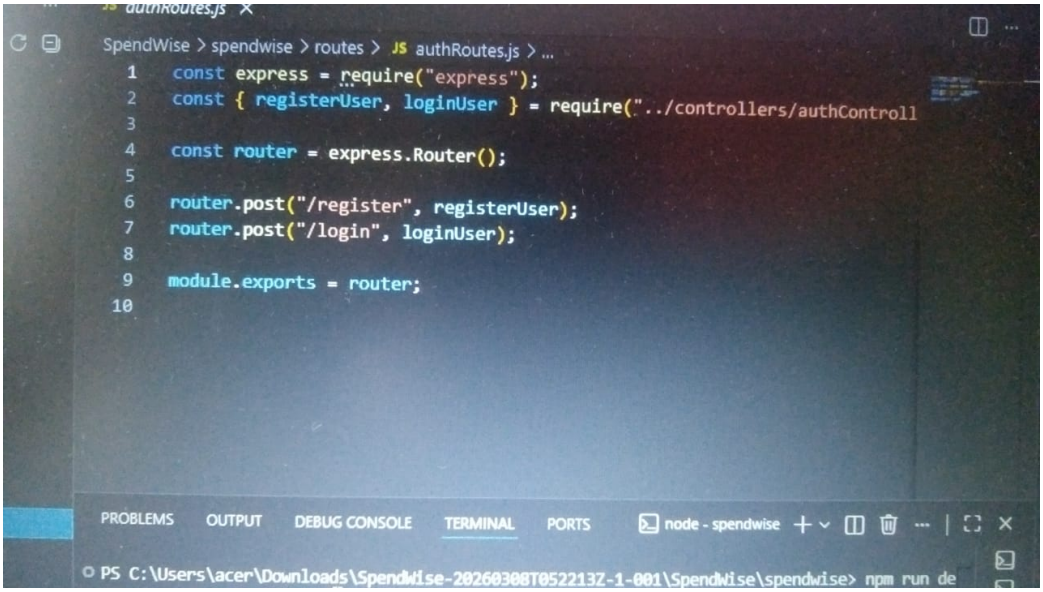
# BACKEND DEVELOPMENT

- Setup Express server
- Create `server.js` file
- Create `.env` file and define PORT and DB URL

```
PORT=5000
MONGO_URI=mongodb+srv://devanshkumar370_db_user:SJ2g23HhsSzuNdSo@cluster0.s3mna8f.mongodb.net/?appName=Cluster0
JWT_SECRET=your_secret_key
```

- Configure middleware (cors, express.json)
- User Authentication:
  - Create Register and Login routes
  - Generate JWT token
  - Protect routes using middleware

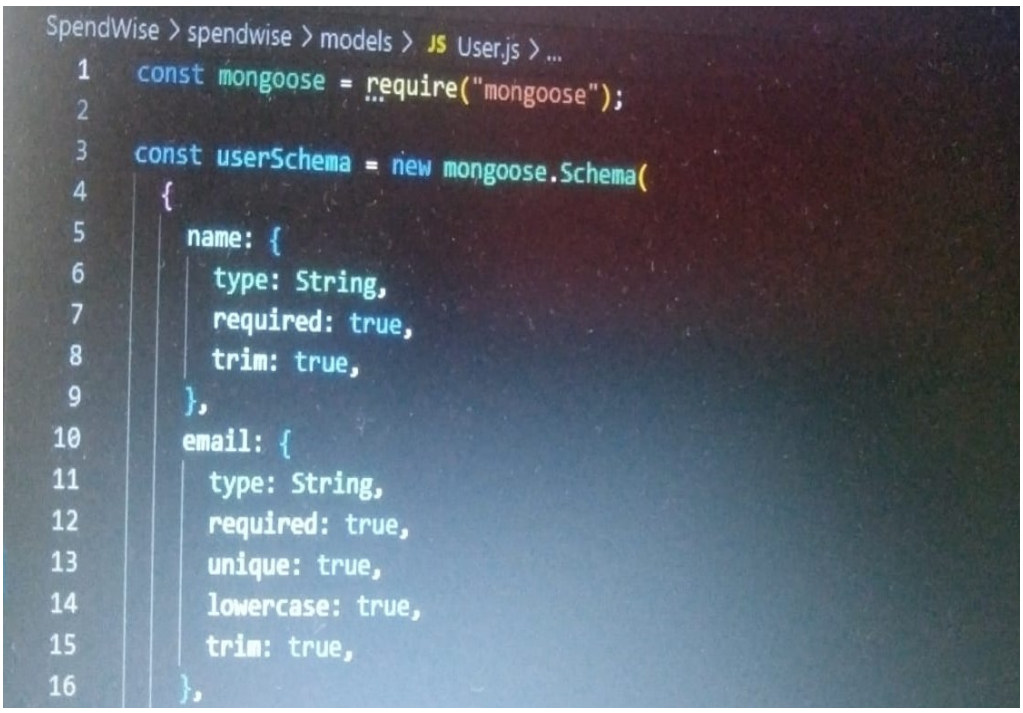
- Define API Routes:
  - Auth routes



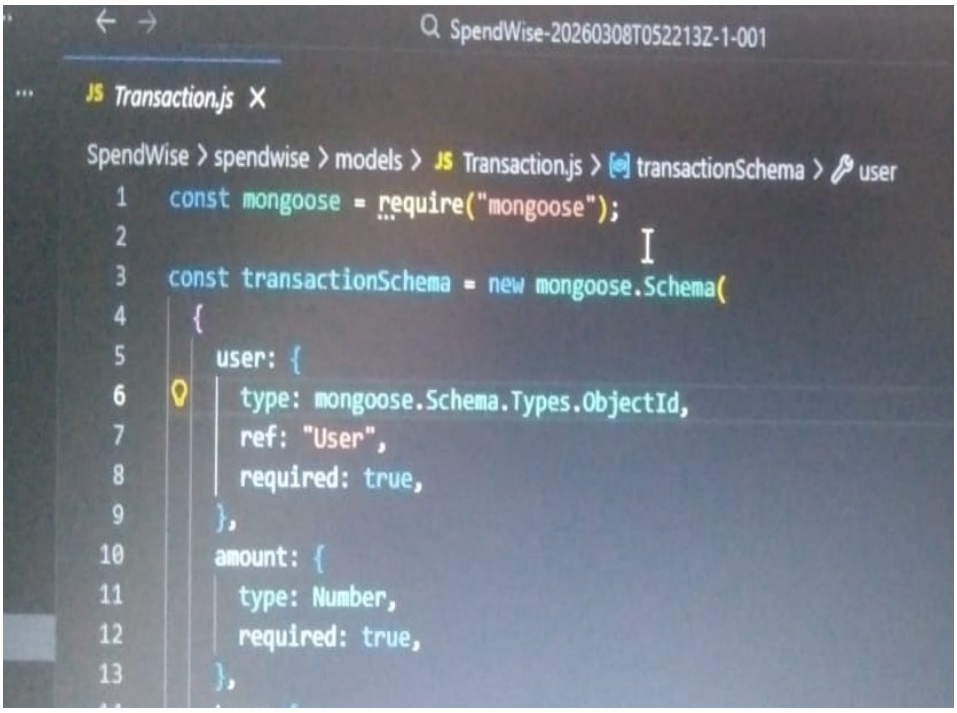

- Transaction CRUD routes

- Implement Data Models:

  - User schema




  - Transaction schema




- Implement CRUD operations
- Add basic error handling

# DATABASE DEVELOPMENT

## Configure MongoDB

- Use MongoDB Atlas or local MongoDB
- Store connection URL in `.env` file
- Connect using Mongoose

## Create Database Connection

- Create a config file for DB connection

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
```

- Use `mongoose.connect()`
- Handle success and error cases

## Create Schema and Models

User Model:

- Name
- email
- password

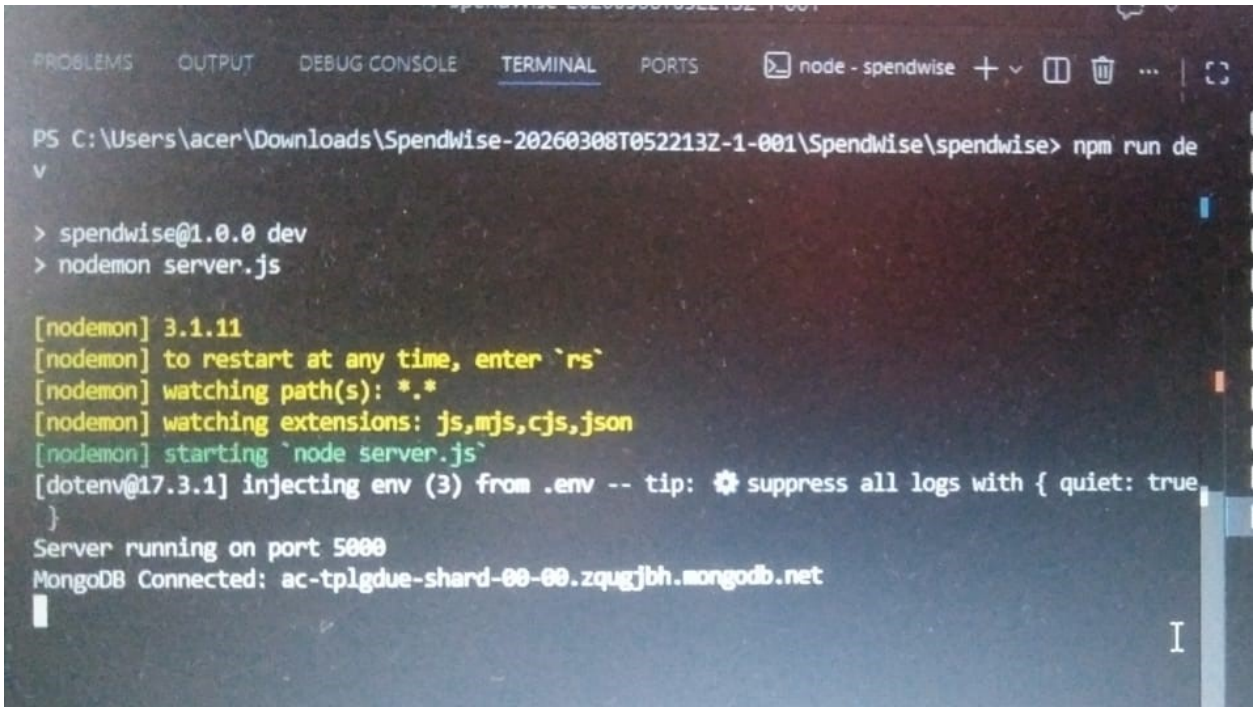
Data / Transaction Model:

- userId
- title / description
- amount / value
- timestamps

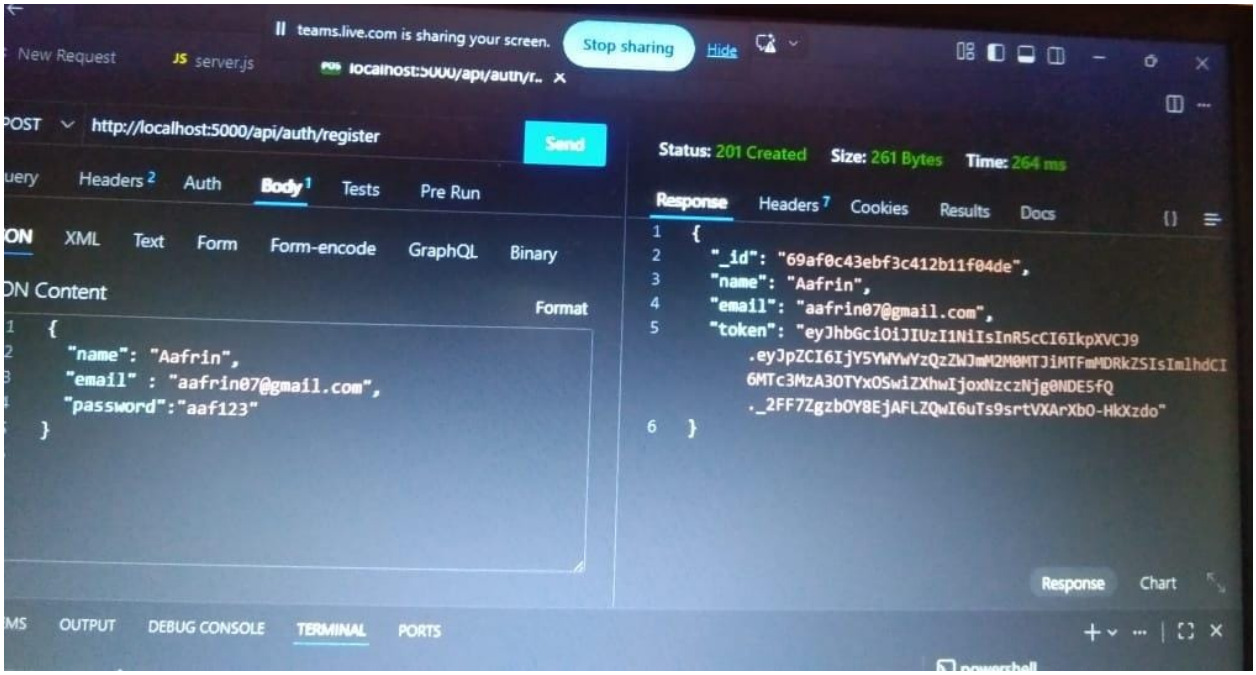
# PROJECT EXECUTION

# Project Execution Steps:

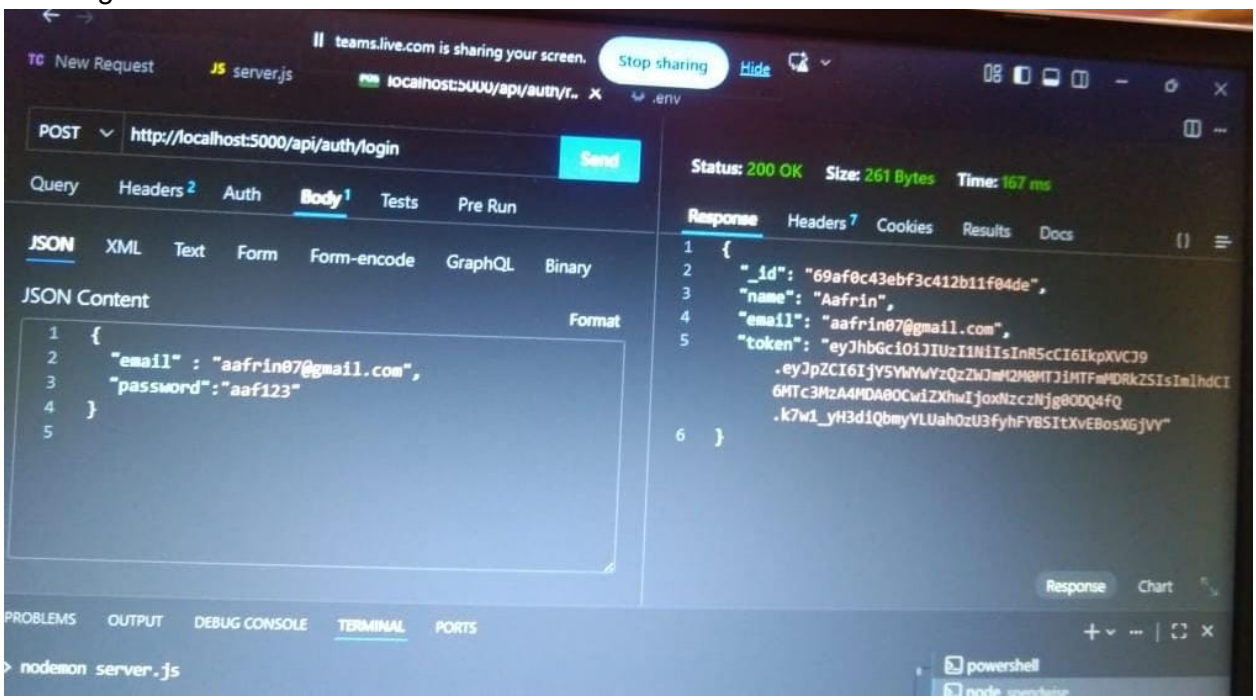
1. Start MongoDB
2. Run backend server



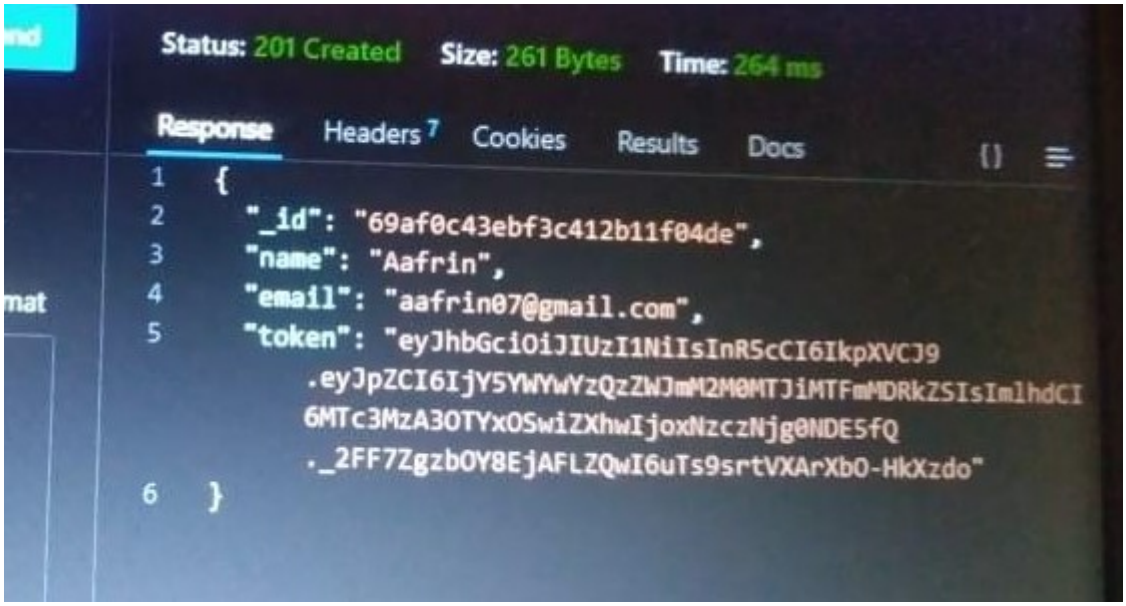

3. Test Register API




4. Test Login API



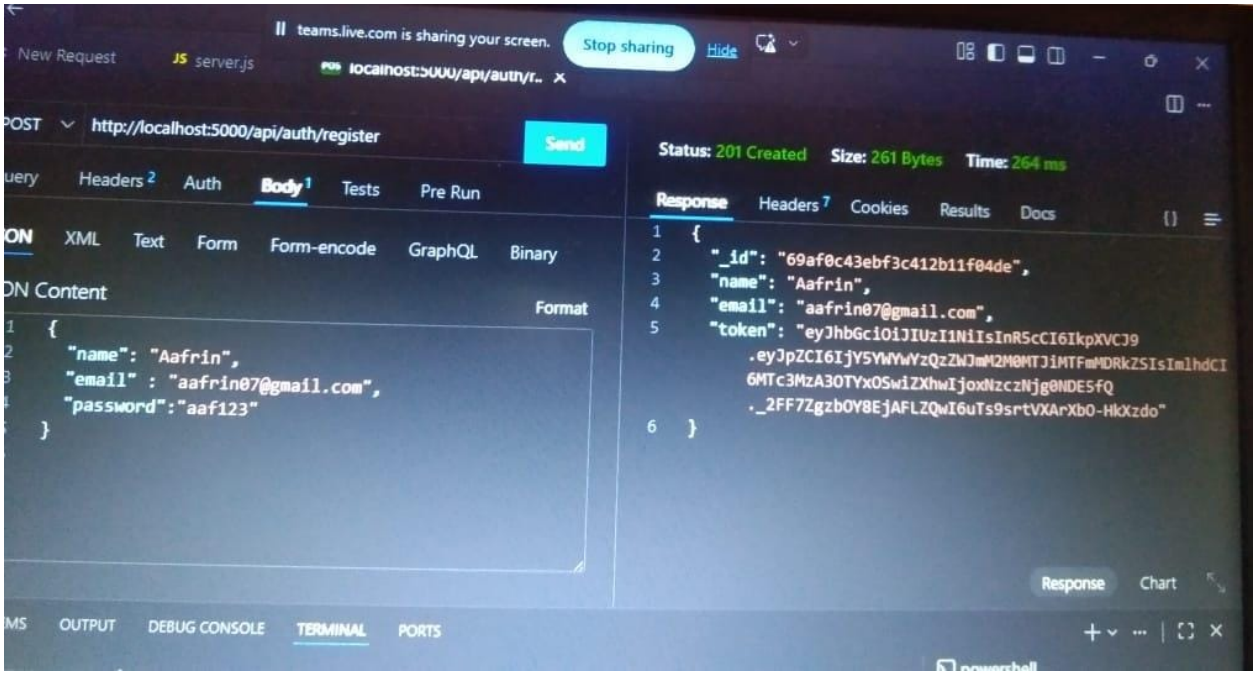

5. Copy JWT token



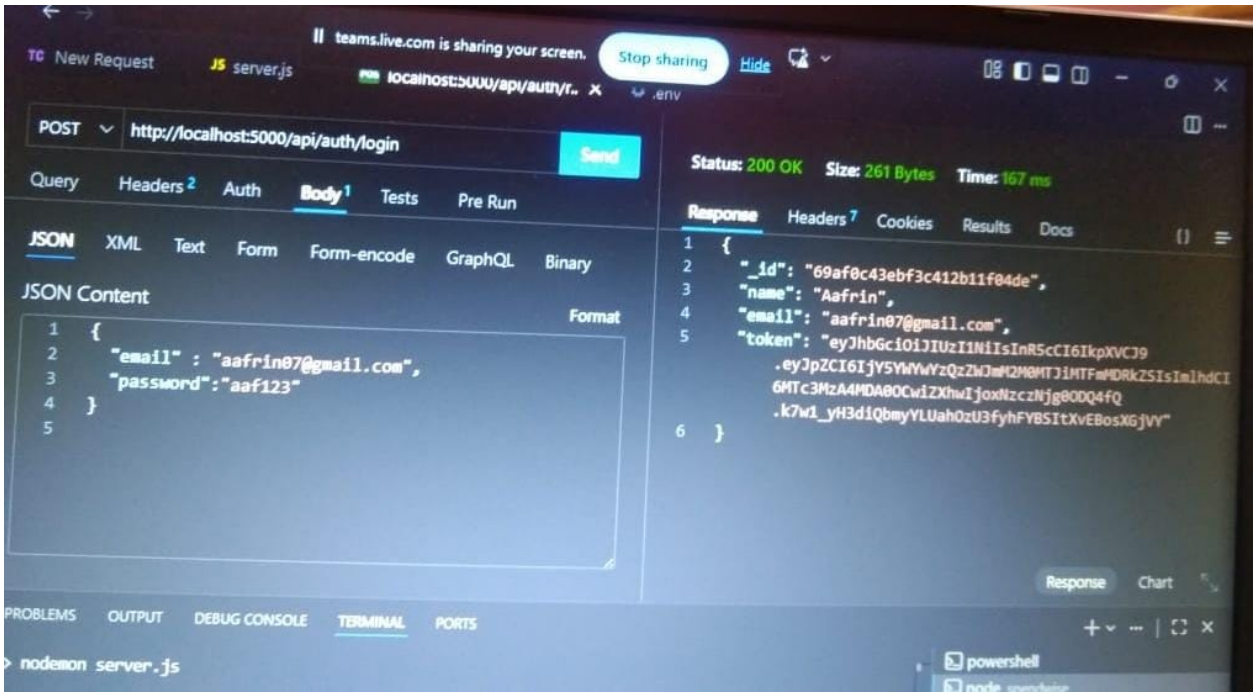

6. Use token to access protected CRUD APIs
7. Perform Create, Read, Update, Delete operations

## OUTPUT SCREENSHOT

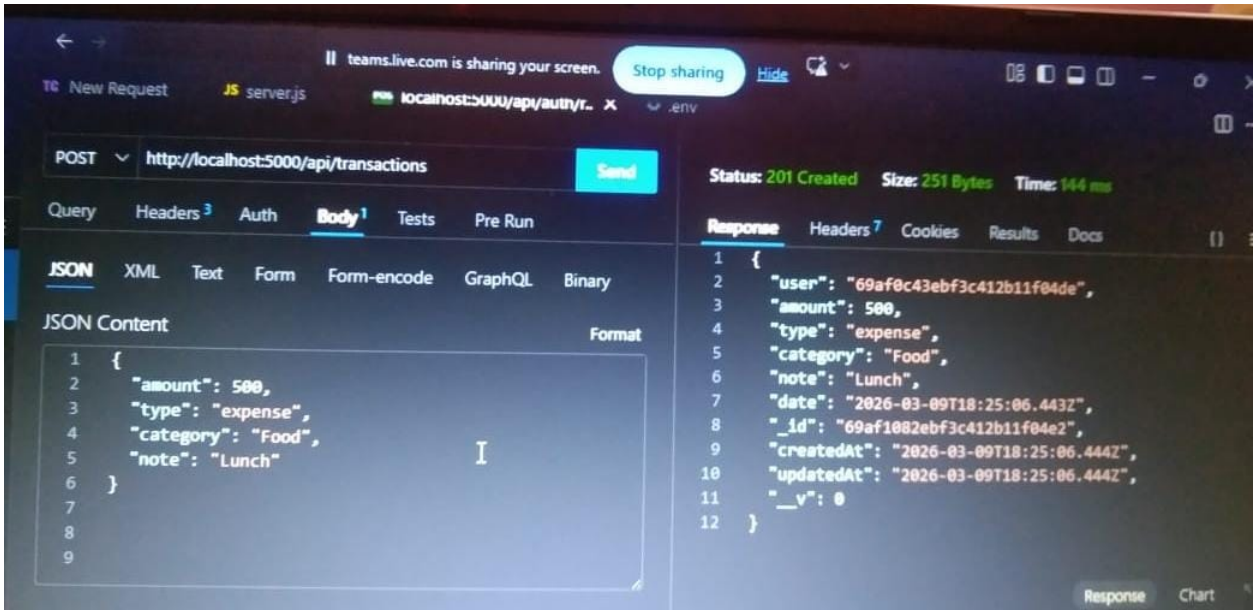
1. Register API



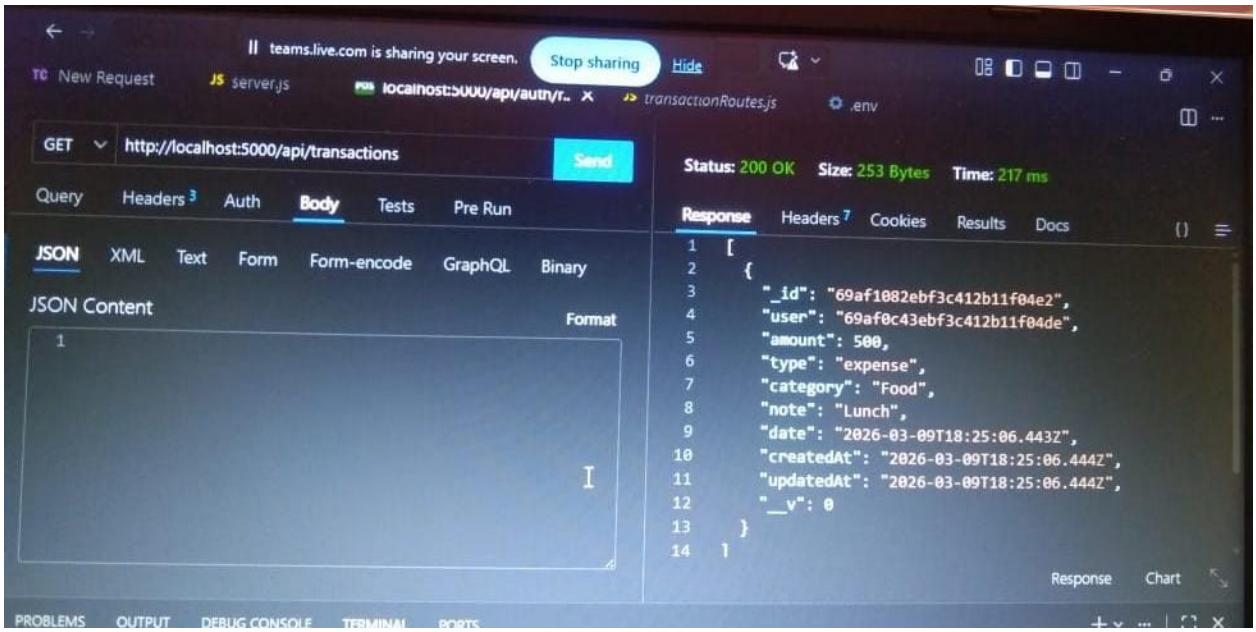

2. Login API



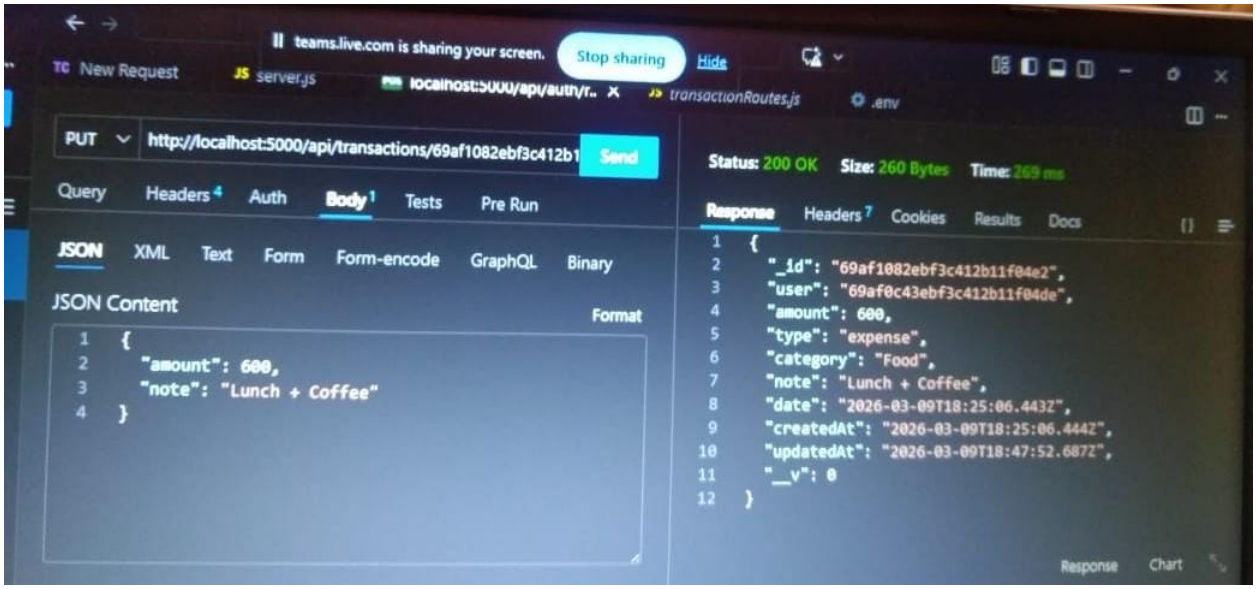

3. Create Transaction



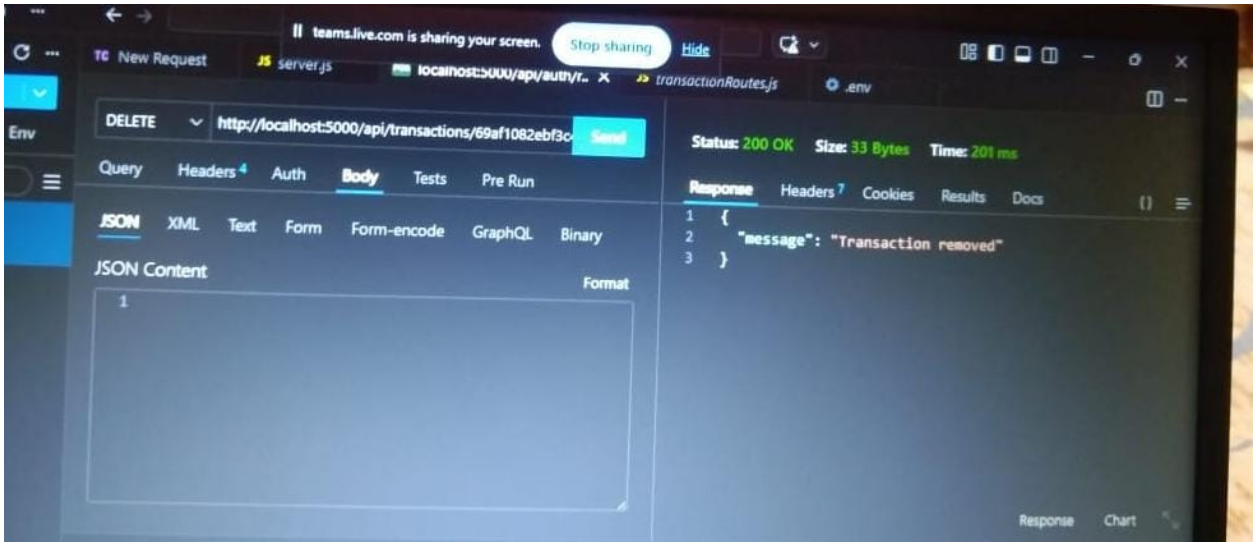

4. Get information




5. Update API




6. Delete API




## Project Demo Video:

https://drive.google.com/file/d/1uFpGW9JEt1UltPVyz_iwWyvK8xuXdPJx/view?usp=drive sdk